

eMenu Cambodia

Server Setup Guide

Complete step-by-step deployment & configuration reference

Domain: emenu-cambodia.com • Server IP: 167.172.88.194 • OS: Ubuntu Linux

Last Updated: June 2026

Architecture Overview

All services run in Docker on a single Linux VPS

Component	Domain / Access	Port	Technology
Nginx Reverse Proxy	All public traffic	80 / 443	Nginx + Wildcard SSL (acme.sh)
Owner Frontend	emenu-cambodia.com	3000	Next.js 14 standalone
Client Frontend	*.emenu-cambodia.com	3001	Next.js 14 standalone
Spring Boot Backend	api.emenu-cambodia.com	7070	Java 21 + Spring Boot 3
PostgreSQL 17	Internal only	5432	Primary database
Redis 7	Internal only	6379	Cache (6 h TTL)

*SSL: Wildcard certificate *.emenu-cambodia.com issued by Let's Encrypt via acme.sh + GoDaddy DNS API. Auto-renews every ~60 days via cron.*

Server Information & Directory Structure

Server Details

VPS Provider	DigitalOcean / any Linux VPS
Server IP	167.172.88.194
OS	Ubuntu Linux
Domain	emenu-cambodia.com (GoDaddy DNS)
Nameservers	ns47.domaincontrol.com / ns48.domaincontrol.com
SSH Access	ssh root@167.172.88.194

Directory Structure on Server

```
/opt/apps/
■■■ emenu-backend/
■   ■■■ e_menu_cambodia.jar    ← Spring Boot fat-jar (built from source)
■   ■■■ Dockerfile
■   ■■■ docker-compose.yml
■■■ owner-frontend/
■   ■■■ Dockerfile            ← Next.js standalone build, PORT=3000
■   ■■■ docker-compose.yml
■■■ client-frontend/
    ■■■ Dockerfile            ← Next.js standalone build, PORT=3001
    ■■■ docker-compose.yml

/etc/nginx/conf.d/
■■■ emenu-owner.conf          ← emenu-cambodia.com      → :3000
■■■ emenu-api.conf            ← api.emenu-cambodia.com  → :7070
■■■ emenu-client.conf         ← *.emenu-cambodia.com   → :3001

/etc/ssl/emenu-cambodia/      ← SSL certificates (managed by acme.sh)
■■■ fullchain.pem
■■■ key.pem
```

SSL Certificate Setup

Wildcard cert via acme.sh + GoDaddy DNS API

Tool	acme.sh (Let's Encrypt ACME client)
Type	Wildcard *.emenu-cambodia.com + emenu-cambodia.com
DNS Provider	GoDaddy (API Key + Secret — regenerate after each use!)
Cert path	/etc/ssl/emenu-cambodia/fullchain.pem
Key path	/etc/ssl/emenu-cambodia/key.pem
Auto-renew	cron job runs daily at 11:44 — renews ~every 60 days automatically

Commands

```
# 1. Install acme.sh (one-time)
curl https://get.acme.sh | sh -s email=phatmenghor19@gmail.com
```

```
# 2. Set GoDaddy API credentials
export GD_Key="YOUR_GODADDY_API_KEY"
export GD_Secret="YOUR_GODADDY_API_SECRET"
```

```
# 3. Issue wildcard certificate
~/.acme.sh/acme.sh --issue --dns dns_gd \
  -d emenu-cambodia.com -d '*.emenu-cambodia.com' \
  --server letsencrypt
```

```
# 4. Install certificate files
mkdir -p /etc/ssl/emenu-cambodia
~/.acme.sh/acme.sh --install-cert -d emenu-cambodia.com \
  --cert-file      /etc/ssl/emenu-cambodia/cert.pem \
  --key-file       /etc/ssl/emenu-cambodia/key.pem \
  --fullchain-file /etc/ssl/emenu-cambodia/fullchain.pem \
  --reloadcmd      "nginx -s reload"
```

```
# 5. Check cert & renewal schedule
```

```
~/acme.sh/acme.sh --list
```

```
# 6. Manual renewal (if needed)
```

```
~/acme.sh/acme.sh --renew -d emenu-cambodia.com --force
```

■ *Always regenerate your GoDaddy API key after use — never commit it to git.*

Nginx Reverse Proxy Configuration

Three virtual hosts — owner (3000), API (7070), client wildcard (3001)

emenu-cambodia.com → :3000 — /etc/nginx/conf.d/emenu-owner.conf

```
server {
    listen 80;
    server_name emenu-cambodia.com www.emenu-cambodia.com;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    server_name emenu-cambodia.com www.emenu-cambodia.com;
    ssl_certificate      /etc/ssl/emenu-cambodia/fullchain.pem;
    ssl_certificate_key  /etc/ssl/emenu-cambodia/key.pem;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

api.emenu-cambodia.com → :7070 — /etc/nginx/conf.d/emenu-api.conf

```
server {
    listen 443 ssl;
    server_name api.emenu-cambodia.com;
    ssl_certificate      /etc/ssl/emenu-cambodia/fullchain.pem;
    ssl_certificate_key  /etc/ssl/emenu-cambodia/key.pem;
    client_max_body_size 50M;
    location / {
        proxy_pass http://127.0.0.1:7070;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

***.emenu-cambodia.com → :3001 — /etc/nginx/conf.d/emenu-client.conf**

```
server {
    listen 443 ssl;
    server_name *.emenu-cambodia.com;
    ssl_certificate      /etc/ssl/emenu-cambodia/fullchain.pem;
    ssl_certificate_key  /etc/ssl/emenu-cambodia/key.pem;
    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

# Test config and reload
nginx -t && nginx -s reload
```

Note: emenu-client.conf must load AFTER emenu-owner.conf so the wildcard does not catch the exact domain.

Backend Deployment

Spring Boot — build jar → copy to server → docker build & run

1. Build JAR (Docker Maven — no JDK needed on dev machine)

```
docker run --rm -v $(pwd):/app -w /app maven:3.9-eclipse-temurin-21-alpine mvn clean package -DskipTests
# Output: menu-scanner-backend/target/e_menu_cambodia.jar
```

2. Copy JAR to server

```
scp menu-scanner-backend/target/e_menu_cambodia.jar \
    root@167.172.88.194:/opt/apps/emenu-backend/
```

3. Copy Dockerfile & docker-compose to server

```
scp menu-scanner-backend/Dockerfile \
    root@167.172.88.194:/opt/apps/emenu-backend/
scp menu-scanner-backend/docker-compose.server.yml \
    root@167.172.88.194:/opt/apps/emenu-backend/docker-compose.yml
```

4. SSH in → build & start

```
ssh root@167.172.88.194
cd /opt/apps/emenu-backend
docker compose build --no-cache
docker compose up -d
```

5. Verify

```
docker ps                                # STATUS: (healthy)
curl http://localhost:7070/actuator/health
```

■ *application-prod.yaml is baked into the JAR at build time. Rebuild the JAR whenever you change YAML config (JWT secret, Telegram token, DB password, etc.).*

Frontend Deployments

Owner (port 3000) and Client (port 3001) — Next.js 14 standalone

Owner Frontend — port 3000 — /opt/apps/owner-frontend/

```
# Dockerfile (port 3000)
FROM node:20-alpine AS builder
WORKDIR /app
COPY . .
RUN npm ci && npm run build

FROM node:20-alpine
WORKDIR /app
ENV PORT=3000 NODE_ENV=production
COPY --from=builder /app/.next/standalone .
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
CMD ["node", "server.js"]

# Build & run
cd /opt/apps/owner-frontend
docker compose build --no-cache && docker compose up -d
```

Client Frontend — port 3001 — /opt/apps/client-frontend/

```
# Dockerfile (port 3001)
FROM node:20-alpine AS builder
WORKDIR /app
COPY . .
RUN npm ci && npm run build

FROM node:20-alpine
WORKDIR /app
ENV PORT=3001 NODE_ENV=production
COPY --from=builder /app/.next/standalone .
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
CMD ["node", "server.js"]
```

```
# Build & run
cd /opt/apps/client-frontend
docker compose build --no-cache && docker compose up -d
```

■ *NEXT_PUBLIC_API_BASE_URL must be set at BUILD time (not runtime) for Next.js. It is baked into the JS bundle during npm run build.*

Database Setup & Production Init Data

PostgreSQL Connection Details

Host	167.172.88.194 (or DB_HOST environment variable)
Port	5432
Database	e_menu_platform
Username	postgres
DDL auto	update (Hibernate creates/alters tables on startup)
pgAdmin tool	Connect via pgAdmin using the credentials above

Production Init Data (scripts/init-production-data.sql)

Run once on a fresh database after the backend starts. Safe to re-run — all inserts use WHERE NOT EXISTS.

What	Detail
PLATFORM_OWNER role	Global role, no business_id
CUSTOMER role	Used by social auth auto-create
Platform owner user	phatmenghor19@gmail.com / password: 88889999
User profile	Phat Menghor • +855-19-000-019
Role assignment	Platform owner user → PLATFORM_OWNER role
Subscription plans	1 Week (WEEKLY) / 1 Month (MONTHLY) / 1 Year (YEARLY) — all PUBLIC, \$0.00

```
-- Verify after running:
SELECT name, price, status, duration_type FROM subscription_plans WHERE is_deleted = false;
-- Expected: 3 rows (1 Week, 1 Month, 1 Year)

-- If pricing section on website still shows empty → flush Redis cache:
```

```
docker exec -it <redis-container-id> redis-cli FLUSHDB
```

Location Data Migration

Province / District / Commune / Village — source DB (bigint) → production (UUID)

The source database uses bigint PKs. Production uses UUID (BaseUUIDEntity). Production FK links are code-based (province_code, district_code, commune_code) — no UUID mapping needed.

Step 1	Open pgAdmin → source database → Query Tool
Step 2	Run the 4 Part A queries from scripts/migrate-location-data.sql (province, district, commune, village)
Step 3	In Data Output panel → click ↓ Download icon → Download as CSV (save as province.csv, etc.)
Step 4	Open each CSV in Notepad → Ctrl+H → find " → replace with nothing → delete header line → Save As .sql (UTF-8)
Step 5	Open pgAdmin → e_menu_platform (production) → Query Tool
Step 6	Ctrl+O (Open File) → select province.sql → F5 to run
Step 7	Repeat in order: district.sql → commune.sql → village.sql
Step 8	Run Part C verify query — counts must match between source and production

```
-- Part C: Verify counts on production
SELECT 'province', COUNT(*) FROM location_province_cbc WHERE is_deleted=false
UNION ALL SELECT 'district', COUNT(*) FROM location_district_cbc WHERE is_deleted=false
UNION ALL SELECT 'commune', COUNT(*) FROM location_commune_cbc WHERE is_deleted=false
UNION ALL SELECT 'village', COUNT(*) FROM location_village_cbc WHERE is_deleted=false;
```

Telegram Bot Setup

Webhook registration + Login Widget + BotFather domain config

Bot Configuration

Webhook URL	https://api.emenu-cambodia.com/api/v1/telegram/webhook
Group chat ID	-1002784141362
Login domain	emenu-cambodia.com (covers all subdomains automatically)
auto-setup	true (backend registers webhook on startup via application-prod.yaml)

BotFather — Enable Login Widget

- Open Telegram → search @BotFather
- Send /setdomain
- Select your bot
- Enter domain: emenu-cambodia.com
- BotFather replies: 'Domain updated' ✓

Set & Verify Webhook (run once after backend is live)

```
# Set webhook
curl -s 'https://api.telegram.org/bot<YOUR_BOT_TOKEN>/setWebhook' \
  --data-urlencode 'url=https://api.emenu-cambodia.com/api/v1/telegram/webhook'

# Verify webhook
curl 'https://api.telegram.org/bot<YOUR_BOT_TOKEN>/getWebhookInfo'
# Expected: "url": "https://api.emenu-cambodia.com/api/v1/telegram/webhook"
```

Moving to a New Server

Complete checklist — follow in this order

1. Provision Server

- ✓ Get a Linux VPS (Ubuntu)
- ✓ SSH in as root
- ✓ Install Docker: `curl -fsSL https://get.docker.com | sh`
- ✓ Install Nginx: `apt-get install -y nginx`
- ✓ Install `acme.sh`

2. DNS Update

- ✓ Log in to GoDaddy
- ✓ Update A record: `emenu-cambodia.com` → new server IP
- ✓ Update A record: `api.emenu-cambodia.com` → new server IP
- ✓ Wildcard `*.emenu-cambodia.com` → new server IP
- ✓ Wait for DNS propagation (~5-30 min)

3. SSL Certificate

- ✓ Generate new GoDaddy API key + secret
- ✓ Run `acme.sh --issue` command (see SSL Setup page)
- ✓ Install cert to `/etc/ssl/emenu-cambodia/`
- ✓ Verify: `ls /etc/ssl/emenu-cambodia/`

4. Nginx

- ✓ Create `/etc/nginx/conf.d/` config files (3 files)
- ✓ `nginx -t && nginx -s reload`

5. Database

- ✓ Start postgres:17 Docker container
- ✓ Connect in pgAdmin with new server IP
- ✓ Start backend — Hibernate auto-creates all tables
- ✓ Run scripts/init-production-data.sql
- ✓ Run scripts/migrate-location-data.sql (Parts A+B)

6. Backend

- ✓ Build JAR: mvn clean package -DskipTests
- ✓ scp jar + Dockerfile + docker-compose to /opt/apps/emenu-backend/
- ✓ docker compose build --no-cache && docker compose up -d
- ✓ docker ps → verify (healthy)

7. Frontends

- ✓ Copy source to /opt/apps/owner-frontend/ and /opt/apps/client-frontend/
- ✓ docker compose build --no-cache && docker compose up -d (both)

8. Telegram

- ✓ Update webhook: curl setWebhook with new URL (if domain changed)
- ✓ Verify with getWebhookInfo

9. Verify All

- ✓ Open https://emenu-cambodia.com — owner frontend
- ✓ Open https://api.emenu-cambodia.com/actuator/health — backend
- ✓ Open https://any-subdomain.emenu-cambodia.com — client frontend

Quick Reference Commands

Day-to-day server operations

Quick Reference Commands

Day-to-day server operations

[illegible]

```
# Restart a service
docker restart emenu_backend
```

```
# █ View live logs ████████████████████████████████████████████████████████████████████████████████
docker logs -f emenu_backend
docker logs -f --tail 100 emenu_backend
```

[illegible][illegible][illegible][illegible][illegible]

```
# ■■ Test and reload Nginx
nginx -t && nginx -s reload
```

[illegible]

Setup Complete — Service Status Checklist

✓	PostgreSQL 17	Running on :5432 • Database: e_menu_platform
✓	Redis 7	Running on :6379 • Cache TTL: 6 hours for subscription plans
✓	Spring Boot Backend	Running on :7070 • Status: healthy • Telegram webhook active
✓	Owner Frontend	Running on :3000 • https://emenu-cambodia.com
✓	Client Frontend	Running on :3001 • https://*.emenu-cambodia.com
✓	Nginx	Routing all 3 domains • SSL termination active
✓	Wildcard SSL	*.emenu-cambodia.com • Auto-renew via acme.sh cron
✓	Telegram Bot	Webhook registered • Login domain: emenu-cambodia.com
✓	Init Data	Platform owner user + 3 subscription plans inserted
✓	Location Data	Province / District / Commune / Village migrated from source DB

emenu-cambodia.com

All systems operational